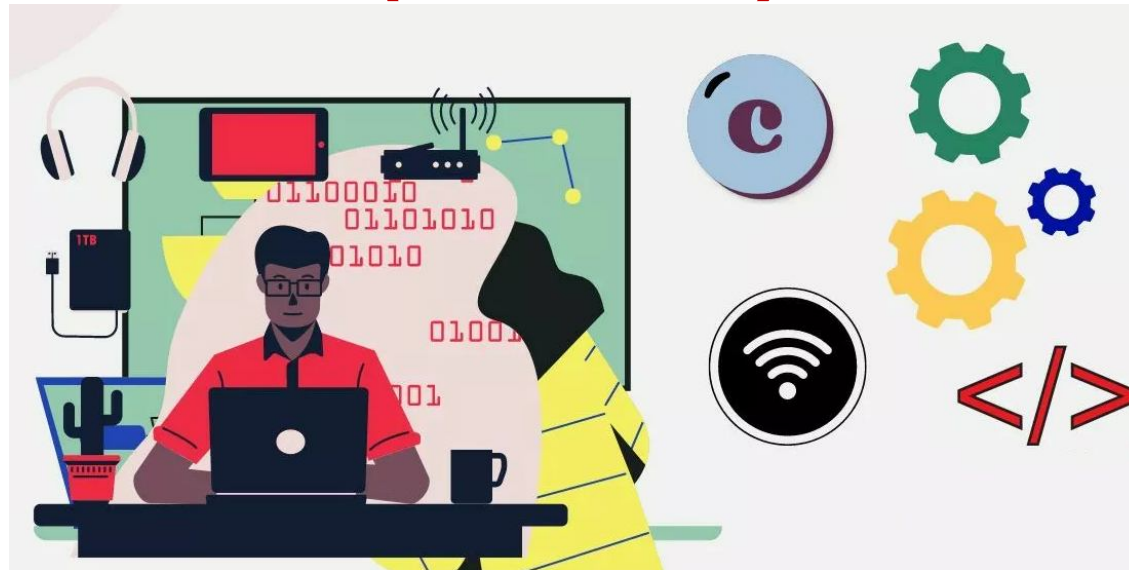




Structured Programming Concepts

(26CSE0105)



Basic Input Output in C

Prepared By: Dr. Punit Soni

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology

Chitkara University, Punjab

Topics to be covered

- ***Basic Input and Output***
- ***Type Conversion***
- ***Escape Sequences***
- ***Coding Standards and Best Programming Practices.***

Basic Input and Output

- **Input and Output** operations are **fundamental in C programming**.
- Enables **communication** between the **program** and the **user**.
- Provided through the **standard input/output** library (**<stdio.h>**).
- Supports **formatted** and **unformatted input/output** operations.

Function	Purpose
printf()	Display output on screen
scanf()	Take input from user
getchar()	Read one character
putchar()	Display one character
gets()/fgets()	Read a line/string
puts()	Display a string

Output Functions (printf())

- The **printf()** function is used to display information on the screen.

```
#include <stdio.h>

int main()
{
    printf("Hello World");
    return 0;
}
```

```
Hello World

...Program finished with exit code 0
Press ENTER to exit console.
```

Output Functions (putchar())

- **putchar()** is used to **displays one character**.

```
#include <stdio.h>

int main()
{
    char ch = 'A';
    putchar(ch);
    return 0;
}
```

```
A
...Program finished with exit code 0
Press ENTER to exit console.□
```

Output Functions (puts())

- **puts()** is used to *display a string*.

```
#include <stdio.h>

int main()
{
    char name[] = "Rajender";
    puts(name);
    return 0;
}
```

```
Rajender

...Program finished with exit code 0
Press ENTER to exit console.□
```

- **puts()** automatically adds a New Line.

Format Specifier's

- **Format specifier** tells **printf()** or **scanf()** what type of data is being used.

Data Type	Format Specifier	Example
int	%d	25
float	%f	25.50
double	%lf	25.5678
char	%c	A
String	%s	Hello

Input Functions (scanf())

- The **scanf()** function is used to take **input from the user**.

```
#include <stdio.h>

int main()
{
    int age;

    printf("Enter your age: ");
    scanf("%d", &age);
    printf("Your age is %d", age);

    return 0;
}
```

```
Enter your age: 40
Your age is 40

...Program finished with exit code 0
Press ENTER to exit console.█
```

Input Functions (getchar())

- The **scanf()** function is used to take **input from the user**.

```
#include <stdio.h>

int main()
{
    char ch;

    printf("Enter a character: ");

    ch = getchar();

    printf("You entered: %c", ch);

    return 0;
}
```

```
Enter a character: A
You entered: A

...Program finished with exit code 0
Press ENTER to exit console.
```

Input Functions (gets())

- **gets()** is used to **read a line of input** from standard input into a **character array**.

```
#include <stdio.h>
int main()
{
    char name[50];
    printf("Enter your name: ");
    gets(name);
    printf("Hello %s", name);
    return 0;
}
```

gets() is completely unsafe,
and was entirely removed
from the C11 standard.

```
main.c: In function 'main':
main.c:6:5: warning: 'gets' is deprecated [-Wdeprecated-declarations]
   6 |     gets(name);
     |     ^~~~
In file included from main.c:1:
/nix/store/0zv32kh0zb4s1v4ld6mc99vmzydj9nm9-glibc-2.40-66-dev/include/stdio.h:667:14: note: declared here
 667 | extern char *gets (char *__s) __wur __attribute_deprecated__;
     |             ^~~~
/nix/store/7h3qnwgvkw6z2r8lq4j5mks4l6r5x2cq-binutils-2.44/bin/ld: /tmp/cctX6evQ.o: in function 'main':
main.c:(.text+0x24): warning: the 'gets' function is dangerous and should not be used.
Enter your name: Chitkara
Hello Chitkara
```

Input Functions (fgets())

- **fgets()** is generally preferable for **reading a line/string** from a specified input stream.

```
#include <stdio.h>
int main()
{
    char name[50];
    printf("Enter your name: ");
    fgets(name, sizeof(name), stdin);
    printf("Hello %s", name);
    return 0;
}
```

```
Enter your name: Punit Soni
Hello Punit Soni

...Program finished with exit code 0
Press ENTER to exit console.
```

Escape Sequence

- Used to represent **special characters** that **cannot be written** directly or have a **special meaning** in C.

Escape Sequence	Name	Meaning	Example Code	Output
<code>\n</code>	Newline	Moves cursor to next line	<code>printf("Hello\nWorld");</code>	Hello World
<code>\t</code>	Horizontal Tab	Inserts a tab space	<code>printf("Hello\tWorld");</code>	Hello World
<code>\b</code>	Backspace	Moves cursor one position back	<code>printf("AB\bC");</code>	AC (B is erased)
<code>\a</code>	Alert / Bell	Produces an alert sound	<code>printf("\a");</code>	 (Bell sound)
<code>\\</code>	Backslash	Prints a backslash (\)	<code>printf("\\");</code>	\
<code>\'</code>	Single Quote	Prints a single quote (')	<code>printf("\'");</code>	'
<code>\"</code>	Double Quote	Prints a double quote (")	<code>printf("\"Hello\"");</code>	"Hello"
<code>\0</code>	Null Character	Marks the end of a string	<code>char s[] = "Hi\0";</code>	Hi (string ends)

Escape Sequence (example)

```
#include <stdio.h>

int main()
{
    printf("Name:\tRaman\n");
    printf("Age:\t22\n");
    printf("He said: \"Hello\"\n");
    printf("Path: C:\\Users\\Raman");
    return 0;
}
```

```
Name:   Raman
Age:    22
He said: "Hello"
Path: C:\Users\Raman

...Program finished with exit code 0
Press ENTER to exit console.
```

Coding Standards and Best Programming Practices



1 Use Meaningful Names

- Choose descriptive names for variables, functions and files.
- Avoid single-letter names (except for simple loops).

```
int count; // good
```

2 Follow a Consistent Style

- Use consistent indentation, braces and spacing.
- Follow a standard coding convention (e.g., Google, ISO, or language-specific).



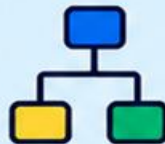
3 Write Comments

- Explain the purpose, not the obvious.
- Use comments for complex logic, assumptions or important notes.



4 Keep Code Modular

- Break the program into small functions or modules.
- Each function should do one task (Single Responsibility Principle).



5 Use Proper Data Types

- Choose the correct data type to save memory and avoid errors.
- Avoid unnecessary type casting.



6 Handle Errors Gracefully

- Check for possible errors and handle exceptions.
- Provide meaningful error messages.



7 Avoid Magic Numbers

- Use constants or named variables instead of hard-coded values.

```
#define MAX_SIZE 100
```

8 Test Your Code

- Test with different inputs and edge cases.
- Use unit testing where possible.



9 Keep It Simple

- Avoid overcomplicating the solution.
- Prefer simple and clear approaches.



10 Document Your Code

- Maintain a README or documentation.
- Explain how to build, run and use the program.



11 Review and Refactor

- Regularly review your code.
- Improve and refactor to make it cleaner and more efficient.





Thank You!

